Sistema de venta de vehículos mediante Programación Orientada a Objetos(POO)

en Python

Presentado por:
Juan Pablo Garzon Ruiz

Gestor del conocimiento:
Juan Camilo Rodriguez Montes

Universidad de Cundinamarca
Facultad de ingeniería
Cadi Programación II
Ubaté,cundinamarca
11/03/2026

Sistema de venta de vehículos mediante Programación Orientada a Objetos(POO) en Python

Introducción

La programación orientada a objetos (POO) permite modelar sistemas del mundo real mediante el uso de clases y objetos. A través de este enfoque es posible representar entidades, sus características y comportamientos dentro del programa.

Este trabajo desarrolló un sistema sencillo para la venta de diferentes tipos de vehículos utilizando Programación Orientada a Objetos(POO). El sistema permite presentar vehículos como carros y motos, mostrando características principales como son la marca, modelo, línea, velocidad y características propias de cada tipo de vehículo.

El objetivo de este programa es demostrar el uso de conceptos de POO como las clases, los objetos, la herencia, el encapsulamiento y los métodos.

Diseño del sistema

El sistema fue diseñado utilizando tres clases principales

- ➔ Vehículo (clase base)
- ➔ Carro (clase hija)
- ➔ Moto (clase hija)

La clase Vehículo representa las características generales de cualquier vehículo, mientras que Carro y Moto heredan estas características y agregan atributos específicos.

Descripción de cada clase

Clase Vehículo: Esta es la clase principal o clase padre, que contiene los atributos y comportamientos comunes a todos los vehículos.

Atributos:

- ➔ marca: marca del vehículo
- ➔ modelo: modelo del vehículo
- ➔ línea: línea o versión

➔ __velocidad: velocidad del vehículo (atributo privado)

Métodos:

➔ aceleracion(): aumenta la velocidad del vehículo.

➔ mostrar_info(): muestra la información del vehículo.

Figura 1.

```python
class Vehiculo:
    def __init__(self, marca, modelo, linea, velocidad=0):
        self.marca = marca
        self.modelo = modelo
        self.linea = linea
        self.__velocidad = velocidad

    def aceleracion(self):
        self.__velocidad += 1

    def mostrar_info(self):
        print("informacion del vehículo:")
        print(f"  Marca: {self.marca}")
        print(f"  Modelo: {self.modelo}")
        print(f"  Línea: {self.linea}")
        print(f"  Velocidad: {self.__velocidad}")
```

Clase Carro: Esta clase hereda de Vehiculo y representa los automóviles.

Atributos:

➔ num_puertas: número de puertas del carro.

Métodos:

➔ mostrar_info(): muestra la información del carro incluyendo el número de puertas.

Figura 2.

```python

class Carro(Vehiculo):
    def __init__(self, marca, modelo, linea, num_puertas):
        super().__init__(marca, modelo, linea)
        self.num_puertas = num_puertas

    def mostrar_info(self):
        super().mostrar_info()
        print(f"  numero de puertas: {self.num_puertas}")
        print(" carro")

```

Clase Moto: Esta clase también hereda de Vehiculo y representa motocicletas.

Atributos:

➔ cilindrada: capacidad del motor en centímetros cúbicos.

Métodos:

➔ mostrar_info(): muestra la información de la moto incluyendo su cilindraje.

Figura 3.

```python
class Moto(Vehiculo):
    def __init__(self, marca, modelo, linea, cilindrada):
        super().__init__(marca, modelo, linea)
        self.cilindrada = cilindrada

    def mostrar_info(self):
        super().mostrar_info()
        print(f"  Cilindraje: {self.cilindrada} cc")
        print(" moto")
```

Relaciones entre las clases

El sistema utiliza herencia, donde:

➔ Vehiculo es la clase padre.

➔ Carro y Moto son clases hijas.

Esto permite reutilizar el código de la clase base y agregar características específicas a cada tipo de vehículo.

También se utiliza encapsulamiento, ya que el atributo __velocidad se declara como privado para proteger su acceso directo.

Justificación del diseño

Se escogió este diseño porque permite representar distintos tipos de vehículos sin repetir código. La herencia facilita que las clases hijas reutilicen los atributos y métodos de la clase Vehiculo, mientras agregan características propias.Este permite que el sistema sea más fácil de mantener.

Implementación

Clases programadas:El programa fue implementado en Python utilizando tres clases principales:

- ➔ Vehiculo
- ➔ Carro
- ➔ Moto

Cada clase contiene atributos y métodos que representan el comportamiento del sistema.

Creación de objetos

Dentro del programa se crean varios objetos de las clases Carro y Moto:

Carros:

- ➔ Ford Ranger 2024
- ➔ Toyota Land Cruiser Prado 2023
- ➔ Renault Oroch 2024

Motos:

- ➔ Pulsar N125 2022

- ➔ Yamaha XTZ 150 2023

- ➔ KTM Duke 250 2023

Estos objetos se almacenan en una lista llamada vehiculos.

Figura 4.

```python
vehiculos = []

carro1 = Carro("FORD", "Ranger", "2024", 4)
carro2 = Carro("TOYOTA", "Land Cruiser Prado", "2023", 5)
carro3 = Carro("RENAULT", "Oroch", "2024", 4)
moto1 = Moto("PULSAR", "N125", "2022", 125)
 (variable) moto3: Moto
moto2 = Moto("YAMAHA", "XTZ 150", "2023", 150)
moto3 = Moto("KTM", "Duke 250", "2023", 250)

vehiculos.append(carro1)
vehiculos.append(carro2)
vehiculos.append(carro3)
vehiculos.append(moto1)
vehiculos.append(moto2)
vehiculos.append(moto3)

for transporte in vehiculos:
    transporte.mostrar_info()
    print("_"*20)
```

Interacción entre objetos

El programa utiliza una lista para almacenar todos los objetos creados. Posteriormente se recorre esta lista utilizando un ciclo for, llamando al método mostrar_info() de cada objeto.

Gracias al uso de polimorfismo, cada objeto ejecuta su propia versión del método mostrar_info() dependiendo de si es un carro o una moto.

Funcionamiento general del programa

- ➔ Se definen las clases del sistema.

- ➔ Se crean varios objetos de tipo carro y moto.

➔ Los objetos se almacenan en una lista.

➔ El programa recorre la lista y muestra la información de cada vehículo.

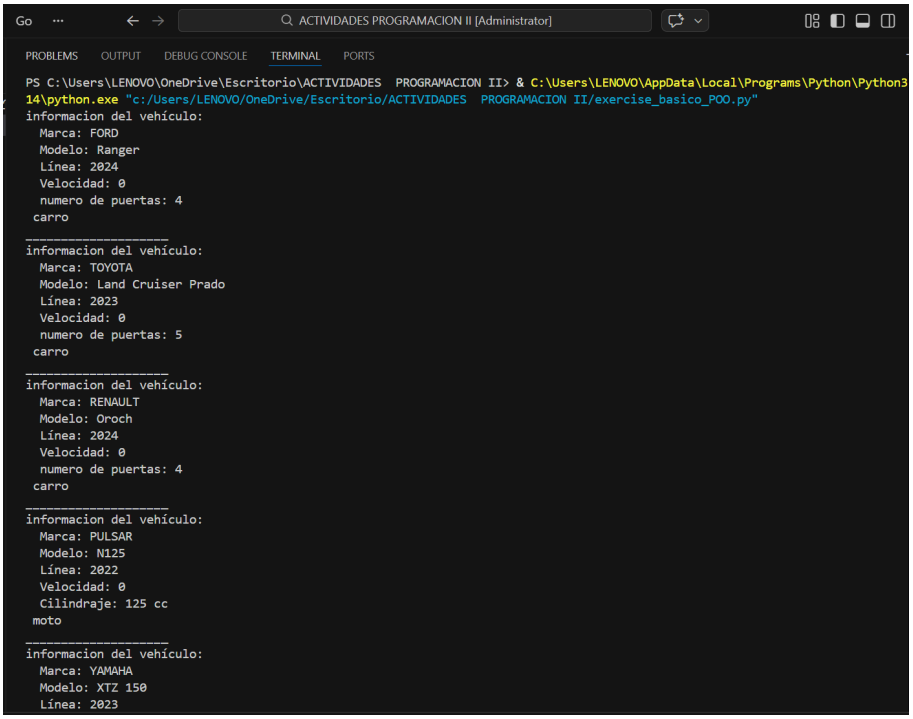
➔ Esto permite visualizar los datos de todos los vehículos registrados en el sistema.

➔

Resultados

Al ejecutar el programa se obtiene una salida en consola donde se muestra la información de cada vehículo registrado, incluyendo sus características principales.

Figura 5.



Conclusiones

➔ Durante el desarrollo de este trabajo se logró comprender mejor el funcionamiento de la programación orientada a objetos y su aplicación en Python.

➔ Se aprendió a utilizar conceptos fundamentales como clases, objetos, herencia, encapsulamiento y polimorfismo, los cuales permiten organizar el código de forma más clara y estructurada.

➔ Una de las principales dificultades fue comprender cómo funcionan las relaciones entre clases y cómo utilizar correctamente los métodos mediante la herencia.

Referencias

Downey, A. (2015). Think Python: How to think like a computer scientist. O'Reilly Media.

Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

Python Software Foundation. (2024). Python documentation. https://docs.python.org/3/